Pregunta 3 — Diseño de Contextos Delimitados para un Sistema de Seguros

Seminario de Arquitecturas de Software

3. Aplicación de DDD y Bounded Context en un Dominio Complejo

En el desarrollo de sistemas de software empresariales de alta complejidad, como una plataforma de seguros de automóviles, uno de los errores más críticos es intentar construir un modelo de dominio unificado y monolítico. Como señalan Richards y Ford (2020), conceptos centrales del negocio como el término "póliza" adquieren significados, reglas y responsabilidades radicalmente opuestas según el departamento que los manipule. Para resolver este desafío, **Domain-Driven Design (DDD)** (Evans, 2003) introduce el concepto de **Bounded Context (Contexto Delimitado)**, el cual delimita explícitamente la aplicabilidad de un modelo de dominio para asegurar que su Lenguaje Ubicuo sea coherente, inequívoco y autónomo en su interior.

3.1. Descomposición del Sistema en Contextos Delimitados

Para estructurar el sistema de seguros de manera desacoplada, se definen cuatro contextos delimitados principales, cada uno con su propia semántica y conjunto de responsabilidades:

1. **Contexto de Suscripción y Gestión de Pólizas:** Enfocado en la evaluación de riesgos, emisión y administración del contrato legal. En este contexto, una "póliza" es un acuerdo formal que agrupa cláusulas, coberturas, vigencias y sumas aseguradas.

2. **Contexto de Facturación de Primas:** Enfocado en la gestión financiera. Aquí, la "póliza" se reduce a una fuente de ingresos, un identificador financiero ligado a un plan de pagos, cuotas, vencimientos y estados de cuenta.

3. **Contexto de Siniestros:** Orientado a la gestión de reclamaciones e indemnizaciones. Para este módulo, la póliza representa el marco jurídico-financiero bajo el cual se evalúa la validez de un incidente reportado.

4. **Contexto de Evaluación de Daños (Ajustadores):** Diseñado para la aplicación móvil de campo. El ajustador no requiere conocer las cláusulas del actuario ni las deudas de facturación; para él, la póliza es simplemente un número de referencia o identificador rápido que valida la preexistencia de cobertura en el lugar del accidente.

3.2. Prevención de la Contaminación del Modelo de Dominio

El uso de una única entidad relacional o un modelo anémico compartido en la base de datos genera una fuerte cohesión indeseada, donde cualquier cambio en la estructura de siniestros rompe los cálculos de facturación. Para evitar esta contaminación del modelo, se aplican los siguientes principios arquitectónicos:

- **Modelos de Dominio Independientes:** Cada Bounded Context posee su propio modelo de objetos en memoria y su propio esquema de persistencia (o tablas dedicadas).
- **Capas de Traducción (*Anti-Corruption Layer - ACL*):** Cuando un contexto necesita interactuar con otro, se implementan traductores explícitos que transforman los conceptos del lenguaje emisor al lenguaje receptor, evitando que los detalles internos de un subsistema contaminen las reglas de negocio de otro.

3.3. Comunicación Desacoplada entre Contextos

Para evitar llamadas síncronas fuertemente acopladas que degraden la disponibilidad del sistema, la comunicación entre los contextos delimitados se resuelve mediante un estilo arquitectónico guiado por eventos:

- **Eventos de Dominio (*Domain Events*):** Cuando ocurre un cambio de estado relevante en el núcleo de un contexto (por ejemplo, se emite una póliza o se declara un siniestro), este emite un evento asíncrono inmutable (ej. `PolizaEmitidaEvent`).
- **Publicación y Suscripción (*Pub/Sub*):** Los contextos interesados (como Facturación) se suscriben de forma independiente a dichos eventos para actualizar sus propios registros de manera desacoplada temporal y espacialmente, eliminando las dependencias directas de código fuente entre los módulos.

3.4. Alineación con Arquitecturas Orientadas al Dominio y Adaptadores de Servicio

Esta partición por contextos delimitados se implementa de manera natural utilizando arquitecturas basadas en capas concéntricas o simétricas, como la **Arquitectura Onion** (Palermo, 2008) o la **Arquitectura Limpia** (Martin, 2017):

- **Protección del Núcleo:** Cada Bounded Context aísla su lógica de negocio pura en el centro de la arquitectura (Entidades y Casos de Uso), blindándola frente a tecnologías externas, frameworks o bases de datos.
- **Adaptadores de Servicio (*Service Adapters*):** Tal como se describe en la *Guía de Arquitectura N-Capas orientada al Dominio con .NET* (De la Torre et al., 2010), la interacción con servicios externos (como pasarelas de pago de terceros o la API de los ajustadores de campo) no debe penetrar el núcleo del dominio. Para ello, se diseñan **Adaptadores de Servicio** en la capa de infraestructura o en el exterior del hexágono. Estos adaptadores actúan como implementaciones de puertos definidos internamente, encargándose de serializar peticiones, traducir protocolos HTTP o transformar formatos de datos externos, garantizando así el cumplimiento estricto de la Regla de Dependencia y una alta mantenibilidad a largo plazo.

Referencias Bibliográficas

- De la Torre Llorente, C., Zorrilla Castro, U., Ramos Barroso, M. A., & Calvarro Nelson, J. (2010). *Guía de Arquitectura N-Capas orientada al Dominio con .NET 4.0 (Borrador)*. Microsoft Ibérica.
- Evans, E. (2003). *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional.
- Martin, R. C. (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- Palermo, J. (2008). *The Onion Architecture*. Consultado en: `https://jeffreypalermo.com/2008/07/the-onion-architecture-part-1/`
- Richards, M., & Ford, N. (2020). *Fundamentals of Software Architecture: An Engineering Approach*. O'Reilly Media.